Pandas Basic Problem-Solving Bank

Cover the major **collections, combinations, filtering patterns, aggregation patterns, joins, and transformations**.

The most useful way is:

> **Problem Statement → Understand the requirement → Data → Step 1 → Step 2 → Final solution → Output → SQL equivalent → Variations**

Below is a comprehensive starter bank.

0. Dataset Used Throughout

We'll use a small dataset so the same problems can gradually become more complex.

```
import pandas as pd
import numpy as np

data = {
    "ID": [101, 102, 103, 104, 105, 106, 107, 108],
    "Name": ["Arun", "Bala", "Chitra", "Deepak", "Esha", "Farhan", "Gita", "Hari"],
    "Age": [21, 25, 30, 22, 28, 35, 26, 31],
    "City": ["Pune", "Mumbai", "Pune", "Chennai",
             "Mumbai", "Pune", "Chennai", "Mumbai"],
    "Department": ["IT", "HR", "IT", "Finance",
                   "HR", "IT", "Finance", "IT"],
    "Salary": [30000, 40000, 55000, 35000,
               45000, 70000, 50000, 65000],
    "Experience": [1, 3, 6, 2, 5, 10, 4, 8],
```

```
    "Status": ["Active", "Active", "Inactive", "Active",
               "Active", "Inactive", "Active", "Active"]
}

df = pd.DataFrame(data)

print(df)
```

PART 1 — DataFrame Basics

Problem 1 — Create a DataFrame

Requirement

Create a DataFrame containing employee information.

Step 1 — Create dictionary

```
data = {
    "ID": [101, 102, 103],
    "Name": ["Arun", "Bala", "Chitra"],
    "Salary": [30000, 40000, 50000]
}
```

Step 2 — Convert dictionary to DataFrame

```
df = pd.DataFrame(data)
```

Step 3 — Display

```
print(df)
```

Problem 2 — Display First 5 Rows

Requirement

Display the first five employees.

```
df.head()
```

Equivalent

df.head(5)

Problem 3 — Display Last 5 Rows

df.tail()

Problem 4 — Find Number of Rows

len(df)

Or:

df.shape[0]

Problem 5 — Find Number of Columns

df.shape[1]

Problem 6 — Find Rows and Columns

df.shape

Example:

(8, 8)

Meaning:

8 rows
8 columns

Problem 7 — Display Column Names

df.columns

Problem 8 — Display Data Types

df.dtypes

Problem 9 — Get Basic Information

df.info()

Problem 10 — Statistical Summary

df.describe()

PART 2 — Column Selection

Problem 11 — Select One Column

Requirement

Display employee names.

df["Name"]

Problem 12 — Select Two Columns

Requirement

Display:

Name
Salary

Solution

df[["Name", "Salary"]]

Problem 13 — Select Three Columns

df[["ID", "Name", "Department"]]

Problem 14 — Select Columns in Different Order

Requirement:

Salary
Name
City

Solution:

df[["Salary", "Name", "City"]]

Important Pattern

df["Name"]

means **one column**.

df[["Name"]]

means **DataFrame containing one column**.

df[["Name", "Salary"]]

means **multiple columns**.

PART 3 — Row Selection

Problem 15 — First Row

df.iloc[0]

Problem 16 — Third Row

df.iloc[2]

Problem 17 — First Three Rows

df.iloc[0:3]

Problem 18 — Rows 2 to 5

df.iloc[1:5]

Problem 19 — Last Row

df.iloc[-1]

PART 4 — Column + Row Combinations

This is where learners start understanding Pandas indexing.

Problem 20

Get:

Name of first employee

Solution

df.iloc[0]["Name"]

Or:

df.iloc[0]["Name"]

Problem 21

Get:

Salary of third employee
df.iloc[2]["Salary"]

Problem 22

Get:

Name and Salary of first three employees
df.iloc[0:3][["Name", "Salary"]]

PART 5 — Filtering

Problem 23 — Employees From Pune

Requirement

Find all employees working in Pune.

Step 1

Identify column:

City

Step 2

Create condition:

df["City"] == "Pune"

Step 3

Apply condition:

df[df["City"] == "Pune"]

Problem 24 — Salary Greater Than 50,000

df[df["Salary"] > 50000]

Problem 25 — Salary Less Than 40,000

df[df["Salary"] < 40000]

Problem 26 — Salary Equal to 50,000

df[df["Salary"] == 50000]

Problem 27 — Age Greater Than 25

df[df["Age"] > 25]

PART 6 — AND Combination

Problem 28

Find employees who:

City = Pune
AND
Salary > 50000

Step 1

First condition:

df["City"] == "Pune"

Step 2

Second condition:

df["Salary"] > 50000

Step 3

Combine:

(df["City"] == "Pune") & (df["Salary"] > 50000)

Step 4

Apply:

```
df[
    (df["City"] == "Pune") &
    (df["Salary"] > 50000)
]
```

PART 7 — OR Combination

Problem 29

Find employees from:

Pune
OR
Mumbai

Solution

```
df[
    (df["City"] == "Pune") |
    (df["City"] == "Mumbai")
]
```

PART 8 — NOT

Problem 30

Find employees who are **not from Pune**.

df[df["City"] != "Pune"]

PART 9 — Multiple Values

Problem 31 — SQL IN Equivalent

Find employees from:

Pune
Mumbai
Chennai

Solution

```
df[
    df["City"].isin(
        ["Pune", "Mumbai", "Chennai"]
    )
]
```

This is the Pandas equivalent of:

WHERE City IN ('Pune', 'Mumbai', 'Chennai')

Problem 32 — NOT IN

```
df[
    ~df["City"].isin(
        ["Pune", "Mumbai"]
    )
]
```

PART 10 — BETWEEN

Problem 33

Find employees whose salary is between:

40,000 and 60,000

Solution

```
df[
    df["Salary"].between(40000, 60000)
]
```

PART 11 — String Filtering

Problem 34

Find employees whose name starts with `A`.

```
df[
    df["Name"].str.startswith("A")
]
```

Problem 35

Find employees whose name contains `a`.

```
df[
    df["Name"].str.contains("a", case=False)
]
```

Problem 36

Find employees from cities beginning with `M`.

```
df[
    df["City"].str.startswith("M")
]
```

PART 12 — Sorting

Problem 37

Sort employees by salary.

df.sort_values("Salary")

Problem 38

Sort salary highest to lowest.

```
df.sort_values(
    "Salary",
    ascending=False
)
```

Problem 39 — Multiple Column Sorting

Sort by:

Department ASC
Salary DESC

Solution

```
df.sort_values(
    ["Department", "Salary"],
    ascending=[True, False]
)
```

This is an important combination.

PART 13 — Top N

Problem 40

Find the highest-paid employee.

```
df.sort_values(
   "Salary",
   ascending=False
).head(1)
```

Problem 41

Find top 3 highest-paid employees.

```
df.sort_values(
   "Salary",
   ascending=False
).head(3)
```

Problem 42

Find the lowest-paid 3 employees.

```
df.sort_values(
   "Salary"
).head(3)
```

PART 14 — Calculated Columns

Problem 43

Create:

Annual Salary

from monthly salary.

Formula

Annual Salary = Salary × 12

Solution

df["AnnualSalary"] = df["Salary"] * 12

Problem 44

Create:

Experience Bonus

where:

Bonus = Salary × 10%
df["Bonus"] = df["Salary"] * 0.10

Problem 45

Create:

Total Compensation
df["TotalCompensation"] = (
    df["Salary"] +
    df["Bonus"]
)

PART 15 — Conditional Columns

Problem 46

Create salary category:

Salary >= 60000 → High
Salary >= 40000 → Medium
Otherwise → Low

Solution

```
conditions = [
    df["Salary"] >= 60000,
    df["Salary"] >= 40000
]

choices = [
    "High",
    "Medium"
]

df["SalaryCategory"] = np.select(
    conditions,
    choices,
    default="Low"
)
```

Problem 47 — np.where

Create:

```
Experienced = Yes
if Experience >= 5
otherwise No
df["Experienced"] = np.where(
    df["Experience"] >= 5,
    "Yes",
    "No"
)
```

PART 16 — Duplicate Handling

Problem 48

Check duplicate rows.

```
df.duplicated()
```

Problem 49

Count duplicate rows.

df.duplicated().sum()

Problem 50

Remove duplicates.

df.drop_duplicates()

PART 17 — Missing Values

Let's create missing values.

```
df.loc[2, "Salary"] = np.nan
df.loc[5, "City"] = np.nan
```

Problem 51 — Find Missing Values

df.isna()

Problem 52 — Count Missing Values

df.isna().sum()

Problem 53 — Find Rows Having Missing Salary

```
df[
    df["Salary"].isna()
]
```

Problem 54 — Fill Missing Salary

df["Salary"] = df["Salary"].fillna(0)

Problem 55 — Fill Missing City

df["City"] = df["City"].fillna("Unknown")

PART 18 — Aggregation

Problem 56 — Total Salary

df["Salary"].sum()

Problem 57 — Average Salary

df["Salary"].mean()

Problem 58 — Maximum Salary

df["Salary"].max()

Problem 59 — Minimum Salary

df["Salary"].min()

Problem 60 — Number of Employees

df["ID"].count()

Or:

len(df)

PART 19 — GROUPBY

Problem 61

Calculate average salary by department.

Step 1

Identify grouping column:

Department

Step 2

Identify calculation:

Average Salary

Step 3

Write:

df.groupby("Department")["Salary"].mean()

Problem 62

Calculate total salary by department.

df.groupby("Department")["Salary"].sum()

Problem 63

Find employee count by department.

df.groupby("Department")["ID"].count()

Problem 64

Find maximum salary by department.

```
df.groupby("Department")["Salary"].max()
```

PART 20 — Multiple Aggregations

Problem 65

For each department calculate:

Employee Count
Average Salary
Minimum Salary
Maximum Salary

Solution

```
df.groupby("Department").agg(
    EmployeeCount=("ID", "count"),
    AverageSalary=("Salary", "mean"),
    MinimumSalary=("Salary", "min"),
    MaximumSalary=("Salary", "max")
)
```

Problem 66 — Group By Two Columns

Calculate average salary by:

Department
City

```
df.groupby(
    ["Department", "City"]
)["Salary"].mean()
```

Problem 67

Calculate total salary by:

City + Department

```
df.groupby(
    ["City", "Department"]
)["Salary"].sum()
```

PART 21 — GROUPBY + FILTER

Problem 68

Find departments whose average salary is greater than ₹50,000.

Step 1

Calculate average:

```
result = (
    df.groupby("Department", as_index=False)
      .agg(
          AverageSalary=("Salary", "mean")
      )
)
```

Step 2

Filter:

```
result[
    result["AverageSalary"] > 50000
]
```

PART 22 — GROUPBY + SORT

Problem 69

Find departments ordered by highest average salary.

```
result = (
    df.groupby("Department", as_index=False)
      .agg(
```

```
        AverageSalary=("Salary", "mean")
    )
    .sort_values(
        "AverageSalary",
        ascending=False
    )
)
```

PART 23 — VALUE COUNTS

Problem 70

Count employees by city.

```
df["City"].value_counts()
```

Problem 71

Count employees by department.

```
df["Department"].value_counts()
```

Problem 72

Count employees by status.

```
df["Status"].value_counts()
```

PART 24 — Combining Filtering + Sorting + Selection

Problem 73

Find the top 3 employees from Pune by salary.

Step 1 — Filter

```
result = df[
    df["City"] == "Pune"
]
```

Step 2 — Sort

```
result = result.sort_values(
    "Salary",
    ascending=False
)
```

Step 3 — Top 3

```
result.head(3)
```

Final

```
df[
    df["City"] == "Pune"
].sort_values(
    "Salary",
    ascending=False
).head(3)
```

PART 25 — Filtering + Multiple Conditions + Sorting

Problem 74

Find active IT employees with salary greater than ₹50,000 and sort by salary descending.

```
result = df[
    (df["Department"] == "IT") &
    (df["Status"] == "Active") &
    (df["Salary"] > 50000)
]

result.sort_values(
    "Salary",
    ascending=False
```

)

PART 26 — Filtering + Calculation

Problem 75

Find employees whose annual salary exceeds ₹600,000.

Step 1

Calculate annual salary:

```
df["AnnualSalary"] = df["Salary"] * 12
```

Step 2

Filter:

```
df[
    df["AnnualSalary"] > 600000
]
```

PART 27 — Groupby + Multiple Aggregations + Sorting

Problem 76

Find the top departments based on total salary.

```
result = (
    df.groupby("Department", as_index=False)
      .agg(
          TotalSalary=("Salary", "sum"),
          AverageSalary=("Salary", "mean"),
          EmployeeCount=("ID", "count")
      )
      .sort_values(
```

```
        "TotalSalary",
        ascending=False
    )
)

print(result)
```

This is a very important **real-world Pandas pattern**.

PART 28 — `apply()`

Problem 77

Create an age category.

Rules:

```
Age < 25      → Young
Age 25–30     → Middle
Age > 30      → Senior
```

Function

```
def age_category(age):

    if age < 25:
        return "Young"

    elif age <= 30:
        return "Middle"

    else:
        return "Senior"
```

Apply

```
df["AgeCategory"] = df["Age"].apply(
    age_category
)
```

PART 29 — Lambda

Problem 78

Create double salary.

```
df["DoubleSalary"] = df["Salary"].apply(
    lambda x: x * 2
)
```

PART 30 — `loc`

Problem 79

Find all IT employees and display only:

```
Name
Salary
df.loc[
    df["Department"] == "IT",
    ["Name", "Salary"]
]
```

PART 31 — `iloc`

Problem 80

Get rows 2–5 and columns 1–3.

```
df.iloc[
    1:5,
    0:3
]
```

PART 32 — Rename Columns

Problem 81

Rename:

Name → EmployeeName
Salary → MonthlySalary

```
df.rename(
    columns={
        "Name": "EmployeeName",
        "Salary": "MonthlySalary"
    }
)
```

PART 33 — String Operations

Problem 82

Convert names to uppercase.

```
df["Name"].str.upper()
```

Problem 83

Convert names to lowercase.

```
df["Name"].str.lower()
```

Problem 84

Find length of names.

```
df["Name"].str.len()
```

Problem 85

Add prefix `EMP-` to employee IDs.

```
df["EmployeeCode"] = (
    "EMP-" + df["ID"].astype(str)
)
```

PART 34 — Combining Two DataFrames

Create another DataFrame:

```
dept = pd.DataFrame({
    "Department": ["IT", "HR", "Finance"],
    "Manager": ["Ravi", "Priya", "Kiran"]
})
```

Problem 86 — INNER JOIN

```
result = df.merge(
    dept,
    on="Department",
    how="inner"
)
```

Problem 87 — LEFT JOIN

```
result = df.merge(
    dept,
    on="Department",
    how="left"
)
```

Problem 88 — RIGHT JOIN

```
result = df.merge(
    dept,
    on="Department",
    how="right"
)
```

Problem 89 — OUTER JOIN

```
result = df.merge(
    dept,
    on="Department",
    how="outer"
)
```

PART 35 — Concatenation

Create:

```
df1 = df.iloc[:4]

df2 = df.iloc[4:]
```

Problem 90

Combine vertically.

```
result = pd.concat(
    [df1, df2]
)
```

Problem 91 — Combine Horizontally

```
result = pd.concat(
    [df1, df2],
    axis=1
)
```

PART 36 — Pivot Table

Problem 92

Find average salary by:

Department
City

```
pd.pivot_table(
    df,
    values="Salary",
    index="Department",
    columns="City",
    aggfunc="mean"
)
```

PART 37 — Ranking

Problem 93

Rank employees based on salary.

```
df["SalaryRank"] = (
    df["Salary"]
    .rank(
        ascending=False,
        method="dense"
    )
)
```

Problem 94

Find top 3 employees.

```
df.nlargest(
    3,
    "Salary"
```

)

PART 38 — `transform()`

Problem 95

Calculate department average salary for every employee.

```
df["DepartmentAverage"] = (
    df.groupby("Department")["Salary"]
      .transform("mean")
)
```

Now every employee gets their department's average salary.

Problem 96

Compare employee salary with department average.

```
df["DepartmentAverage"] = (
    df.groupby("Department")["Salary"]
      .transform("mean")
)

df["AboveDepartmentAverage"] = (
    df["Salary"] >
    df["DepartmentAverage"]
)
```

PART 39 — `shift()`

Problem 97

Create previous employee salary.

```
df["PreviousSalary"] = (
    df["Salary"].shift(1)
)
```

Problem 98

Calculate salary difference from previous row.

```
df["PreviousSalary"] = (
    df["Salary"].shift(1)
)

df["SalaryDifference"] = (
    df["Salary"] -
    df["PreviousSalary"]
)
```

PART 40 — Cumulative Calculations

Problem 99

Calculate cumulative salary.

```
df["CumulativeSalary"] = (
    df["Salary"].cumsum()
)
```

Problem 100

Calculate cumulative salary by department.

```
df["CumulativeDepartmentSalary"] = (
    df.groupby("Department")["Salary"]
      .cumsum()
)
```

🧩 The Combination Matrix

This is the important part for your **"all combinations"** requirement.

Instead of learning 100 unrelated questions, learn how Pandas operations combine.

Combination	Example
Filter + Select	Pune → Name, Salary
Filter + Sort	Pune → highest salary
Filter + Top N	Pune → top 3
Filter + Calculation	Salary > annual threshold
Filter + GroupBy	Active employees → department count
Filter + GroupBy + Sort	Active → department → salary
Select + Sort	Name + Salary → descending
GroupBy + Sum	Department → total salary
GroupBy + Mean	Department → average salary
GroupBy + Count	Department → employee count
GroupBy + Multiple Aggregations	count + min + max + avg
GroupBy + Filter	Average salary > threshold
GroupBy + Sort + Top N	Top departments
GroupBy + Two Columns	City + Department
GroupBy + Transform	Department average per employee
Sort + Rank	Salary ranking
Merge + Filter	Customers with orders
Merge + GroupBy	Customer revenue
Merge + GroupBy + Sort	Top customers
Merge + Calculate	Revenue / profit

String + Filter	Names containing text
String + Transform	Standardize names
Missing + Fill + Calculate	Fill salary → bonus
Date + Extract + GroupBy	Monthly sales
Date + Filter + Sort	Recent transactions
Pivot + Aggregation	Revenue by city/category
Apply + Conditional Logic	Custom classification

🔥 The 15 Most Important Combination Problems

These should become your **core practice set**.

Challenge 1

Find the top 5 highest-paid IT employees.

Filter
↓
Sort
↓
Top N

Challenge 2

Find average salary by city.

GroupBy
↓
Mean

Challenge 3

Find the highest-paid employee in each department.

GroupBy
↓
Sort / idxmax

Challenge 4

Find departments with more than 2 employees.

GroupBy
↓
Count
↓
Filter

Challenge 5

Find active employees earning above their department average.

Filter
↓
GroupBy
↓
Transform
↓
Compare

Challenge 6

Find top 2 employees in every department.

GroupBy
↓
Rank
↓
Filter

Challenge 7

Calculate total salary by city and department.

GroupBy
↓
Two columns
↓
Sum

Challenge 8

Find employees whose salary is above the company average.

Mean
↓
Compare
↓
Filter

Challenge 9

Find the city with the highest average salary.

GroupBy
↓
Mean
↓
Sort
↓
Head(1)

Challenge 10

Find the number of active employees by city.

Filter
↓
GroupBy
↓

Count

Challenge 11

Create salary categories and count employees in each category.

Conditional Column
↓
Value Counts

Challenge 12

Calculate total salary and average salary for each department, then rank departments.

GroupBy
↓
Multiple Aggregations
↓
Rank / Sort

Challenge 13

Join employee data with department manager information.

Merge
↓
Select
↓
Validate

Challenge 14

Find departments where average employee salary is greater than ₹50,000.

GroupBy
↓
Average
↓

Filter

Challenge 15 — Complete Business Problem

> Find the top 3 active IT employees with more than 3 years of experience, earning above the IT department average salary.

Break it down:

```
ACTIVE
  ↓
IT
  ↓
EXPERIENCE > 3
  ↓
CALCULATE IT AVERAGE
  ↓
SALARY > IT AVERAGE
  ↓
SORT SALARY DESC
  ↓
TOP 3
```

Possible solution:

```
it_avg_salary = (
    df.loc[
        df["Department"] == "IT",
        "Salary"
    ].mean()
)

result = df[
    (df["Status"] == "Active") &
    (df["Department"] == "IT") &
    (df["Experience"] > 3) &
    (df["Salary"] > it_avg_salary)
]

result = result.sort_values(
    "Salary",
    ascending=False
).head(3)
```

```
print(result)
```

🧠 The Pandas Mental Model

When you see a problem, don't immediately write code.

Ask:

WHAT DO I NEED?
↓
ROWS?
↓
COLUMNS?
↓
FILTER?
↓
CALCULATION?
↓
GROUP?
↓
JOIN?
↓
SORT?
↓
TOP N?
↓
FINAL OUTPUT?

Then translate each requirement into Pandas.

🏆 Pandas Problem-Solving Ladder

LEVEL 1
Create DataFrame
↓
LEVEL 2
Select Columns
↓

LEVEL 3
Select Rows
↓
LEVEL 4
Filter
↓
LEVEL 5
Multiple Filters
↓
LEVEL 6
Sort
↓
LEVEL 7
Calculated Columns
↓
LEVEL 8
Conditional Columns
↓
LEVEL 9
Aggregation
↓
LEVEL 10
GROUPBY
↓
LEVEL 11
GROUPBY + Multiple Aggregations
↓
LEVEL 12
MERGE
↓
LEVEL 13
GROUPBY + MERGE
↓
LEVEL 14
TRANSFORM
↓
LEVEL 15
RANK / SHIFT
↓
LEVEL 16
PIVOT
↓
LEVEL 17
DATE/TIME

↓
LEVEL 18
DATA CLEANING
↓
LEVEL 19
MULTI-STEP BUSINESS PROBLEMS
↓
LEVEL 20
SQL vs PANDAS COMPETITION

This gives your trainees a progression from **simple syntax** → **individual operations** → **combinations** → **business logic** → **Data Engineering scenarios**, which is much more effective than teaching Pandas functions one by one.